



Mangaldeep Consulting

Claim Survey Management System

A Next.js and MySQL platform for insurance surveys



Presented by

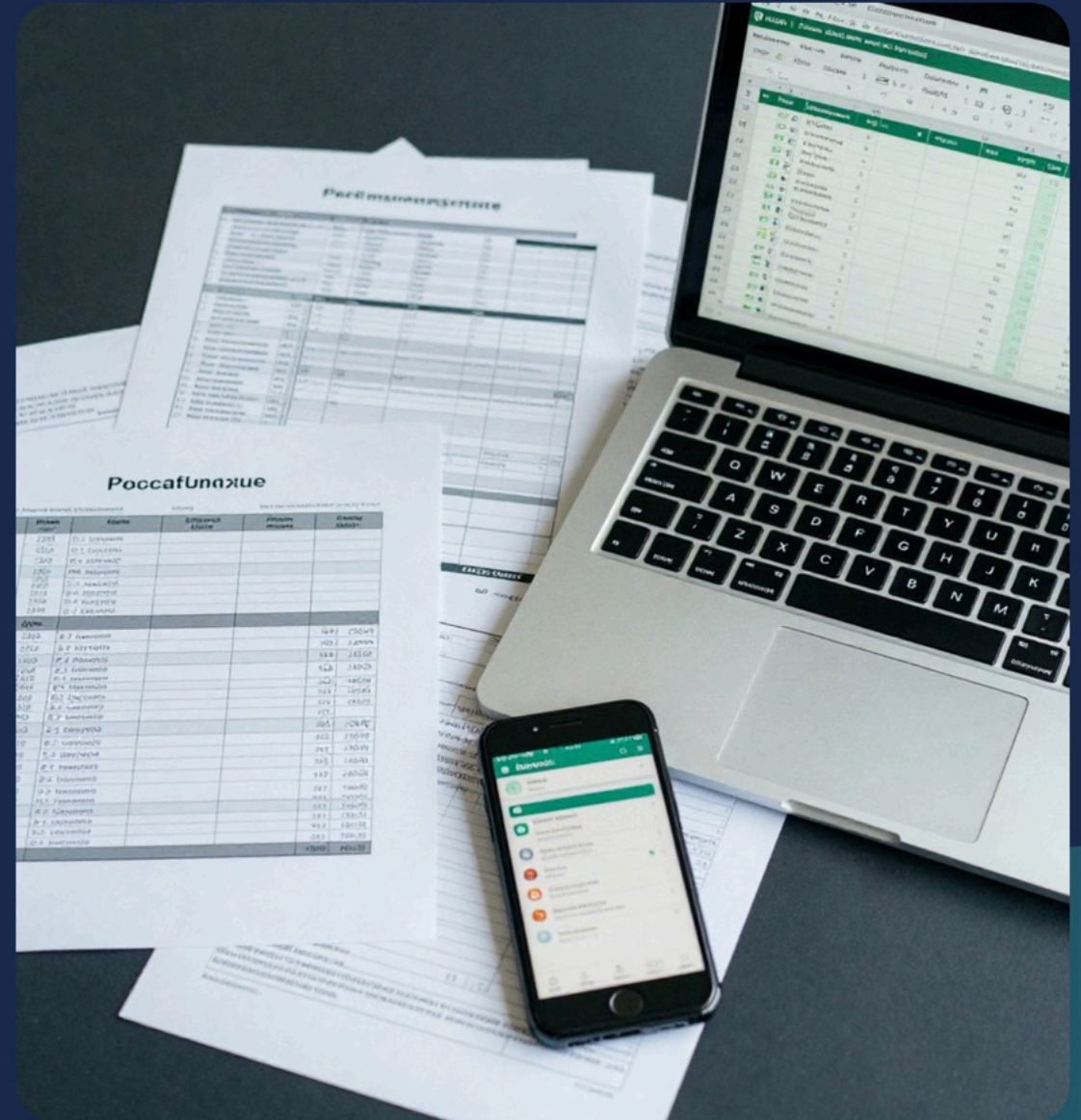
Shaksham Karki and Bikram Timalisina

DATA 210 · Database Design and Analytics

6 October 2026

Problem Statement

- Motor, marine and industrial claims are managed in one shared Excel sheet.
- Site visits are arranged over WhatsApp, damage reports sit in Word files, and fees are tracked in a separate sheet.
- Insurer and client details are retyped for every claim, and totals are added by hand, so mistakes creep in.
- Duplicate claims slip through, and there is no quick way to see open claims or unpaid fees.



Current Process Issues

Data and documentation

- Insurer and client details re-entered per claim
- Damage reports split across Word files
- Duplicate claim entries occur

Tracking and visibility

- Fee totals calculated by hand
- No single view of open claims
- Unpaid fees hard to track



Requirements & Business Rules

Validated with Uttam Regmi, Proprietor & Senior Surveyor, Mangaldeep Consulting.

Functional

- Register a claim using existing insurer, client and surveyor
- Log each site visit with date and findings
- One invoice per claim, record payments
- List open claims and unpaid fees
- Flag possible duplicate claims

Non-functional

- Staff must log in (admin / staff roles)
- Every change kept in an audit trail
- Runs in a browser on free software (MySQL + Next.js)

Scope

- In: claims, visits, invoices, reports
- Out: photos, policy data, accounting

Business rules

- A claim has exactly one insurer, client and surveyor
- A claim can have many visits, but one invoice
- Claimed amount and fee must be above 0
- Amount paid can never exceed the fee
- Surveyor licence numbers are unique
- Records in use by a claim cannot be deleted

Claim Lifecycle

01 Register

Add the claim once, picking an existing insurer, client and surveyor. Nothing is retyped.

02 Assign

Assign a surveyor and log each site visit in the system, not over WhatsApp.

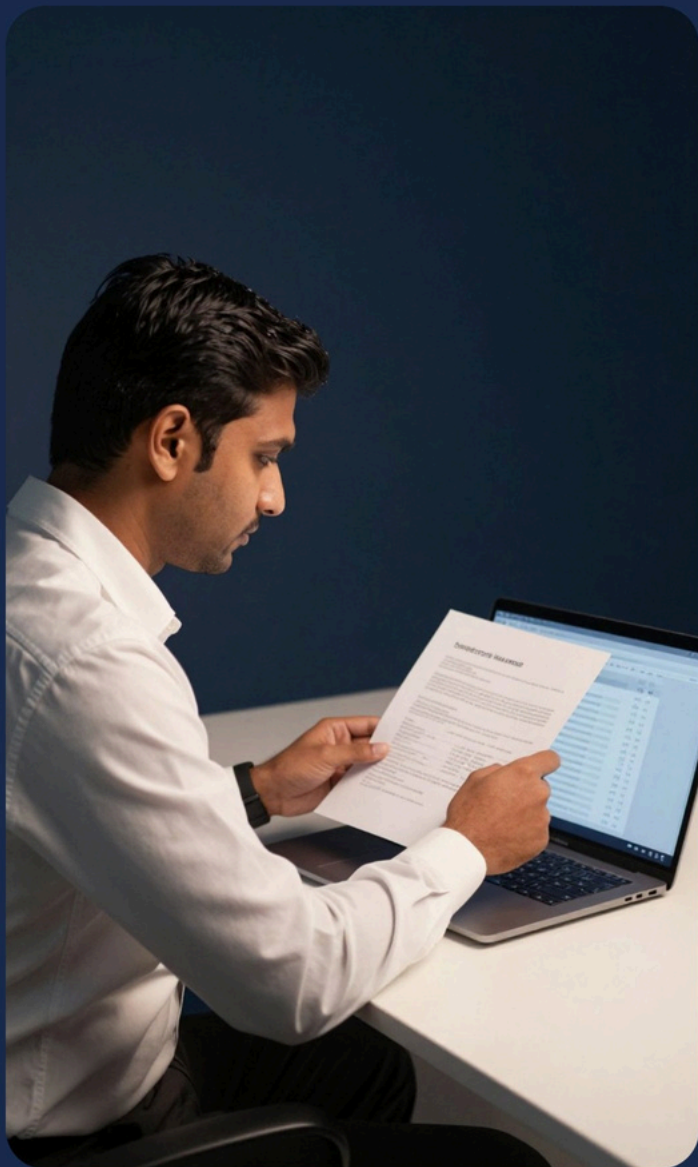
03 Survey

Record the findings of every visit digitally, replacing scattered Word files.

04 Invoice

Raise one invoice per claim and record payments, so unpaid fees are never missed.

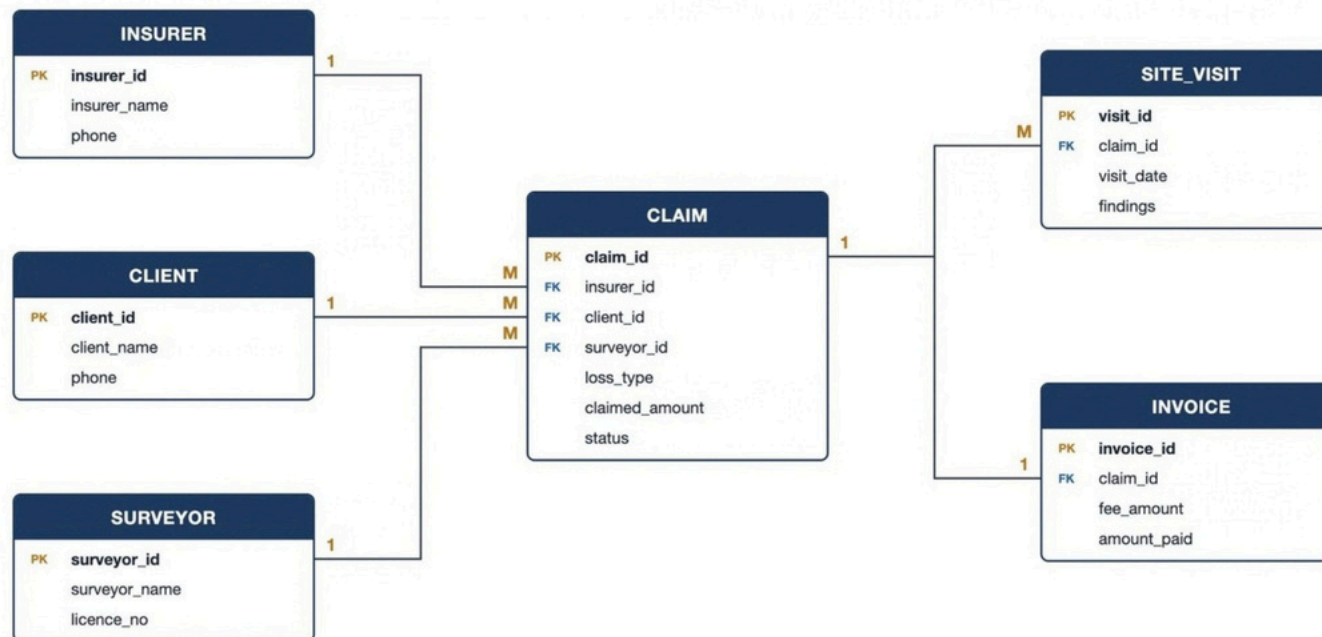




Database Architecture

Claim Survey Management Database: ER Diagram

Mangaldeep Consulting Pvt. Ltd. · DATA 210 · Shaksham Karki, Bikram Timalsina, Rosis KC



PK = primary key FK = foreign key 1 = one M = many

Six linked tables. CLAIM sits at the centre, joined to its insurer, client and surveyor by foreign keys.

Attributes and Relationships

Table	Columns
INSURER	insurer_id (PK), insurer_name, phone
CLIENT	client_id (PK), client_name, phone
SURVEYOR	surveyor_id (PK), surveyor_name, licence_no
CLAIM	claim_id (PK), insurer_id, client_id, surveyor_id (FK), loss_type, claimed_amount, status
SITE_VISIT	visit_id (PK), claim_id (FK), visit_date, findings
INVOICE	invoice_id (PK), claim_id (FK), fee_amount, amount_paid

Relationships

Insurer	1:M	Claim
Client	1:M	Claim
Surveyor	1:M	Claim
Claim	1:M	Site visit
Claim	1:1	Invoice

Data Dictionary

Column	Type	Rule
*_id	INT UNSIGNED	PK, auto number
insurer_id, client_id, surveyor_id	INT UNSIGNED	FK in CLAIM, NOT NULL
insurer_name	VARCHAR(120)	NOT NULL, UNIQUE
licence_no	VARCHAR(30)	NOT NULL, UNIQUE
loss_type	ENUM	Marine / Motor / Industrial
claimed_amount	DECIMAL(12,2)	CHECK > 0
status	ENUM	Open / Surveyed / Closed
visit_date	DATE	NOT NULL
fee_amount	DECIMAL(10,2)	CHECK > 0
amount_paid	DECIMAL(10,2)	CHECK $0 \leq \text{paid} \leq \text{fee}$

Why MySQL?

- Free and runs on an office PC
- Enforces PK, FK, UNIQUE and CHECK rules
- Supports views and triggers for reports and audit
- One-command backup with mysqldump

Login and audit use 3 support tables (app_user, user_session, audit_log), kept outside the business ERD.

Normalization

Four steps that turn Mangaldeep's single claims sheet into six linked tables, each fact stored once.



Unnormalized Form (UNF)



The Excel sheet today: one wide row holds the whole claim.

claim_id	insurer_name	insurer_phone	client_name	visit1_date	visit2_date	fee_amount	...
C-01	Everest Shield	01-4412345	Bagmati Traders	2026-06-04	2026-06-18	37,000	...
C-04	Everest Shield	01-4412345	Kathmandu Freight	2026-07-06	2026-07-24	52,000	...
C-09	Everest Shield	01-4412345	Himal Cement	2026-09-15	—	—	...

7 of the 16 columns shown. Highlighted cells repeat on every claim. Dashed columns are a repeating group.



Insertion

A new surveyor can't be saved without a claim



Update

One phone change means editing every claim row



Deletion

Delete a claim and the client's details go too

First Normal Form (1NF)



Rule: one value per cell, no repeating groups, and a primary key for every row.

BEFORE · VISITS AS COLUMNS

claim_id	visit1_date	visit2_date
C-01	2026-06-04	2026-06-18



AFTER · ONE ROW PER VISIT

claim_id	visit_date	insurer_name	findings
C-01	2026-06-04	Everest Shield	Water in 40 cartons
C-01	2026-06-18	Everest Shield	Loss assessed 31%

New primary key

claim_id + visit_date

One claim can now have any number of visits.

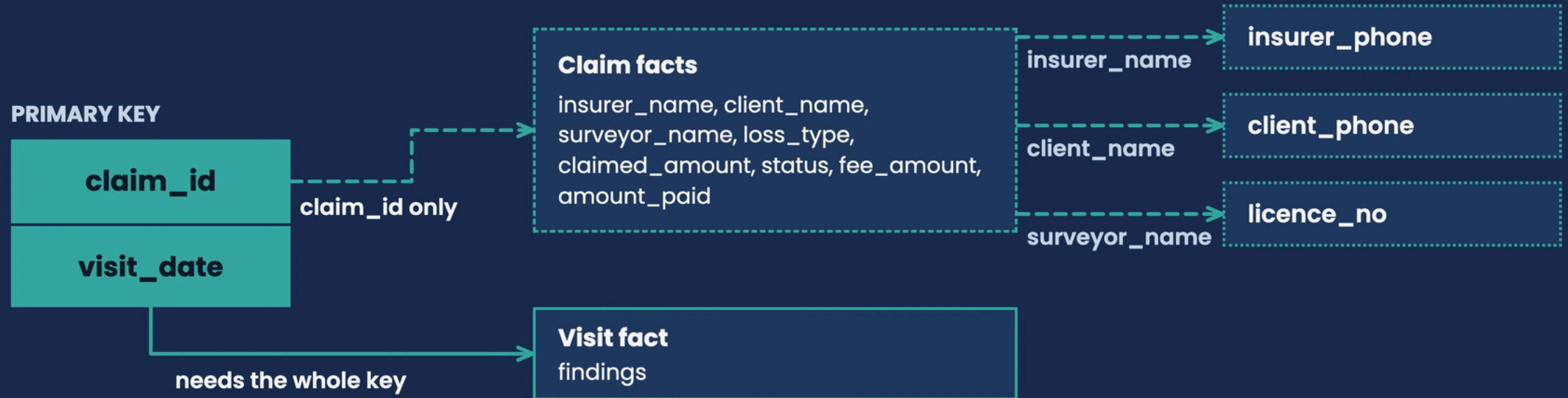


Still a problem: claim details repeat on every visit row, and a claim with no visit yet cannot be stored, because visit_date is part of the key.

Functional Dependencies



$A \rightarrow B$ means knowing A fixes B. Found in the 1NF table, CLAIM_VISIT.



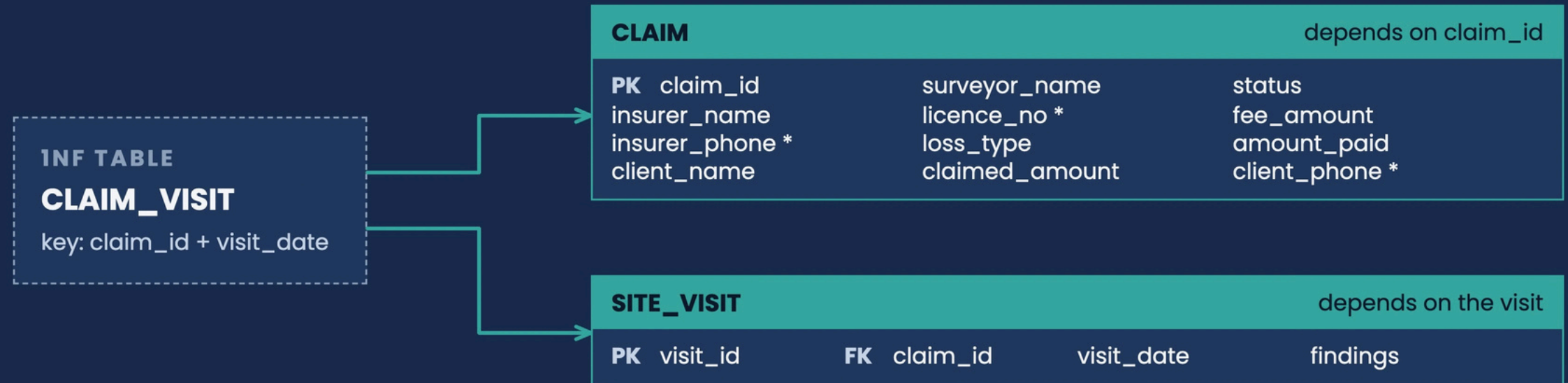
— **Full:** fine - - - **Partial:** breaks 2NF **Transitive:** breaks 3NF

Assumption for this step: each insurer, client and surveyor name identifies one record. 3NF replaces names with IDs, so the final design does not rely on it.

Second Normal Form (2NF)



Rule: every non-key column depends on the whole key. Remove partial dependencies.



* insurer_phone, client_phone and licence_no still depend on a non-key column, which breaks 3NF. The new key visit_id allows two visits on the same day.

Third Normal Form (3NF)



Rule: no non-key column depends on another non-key column. Remove transitive dependencies.

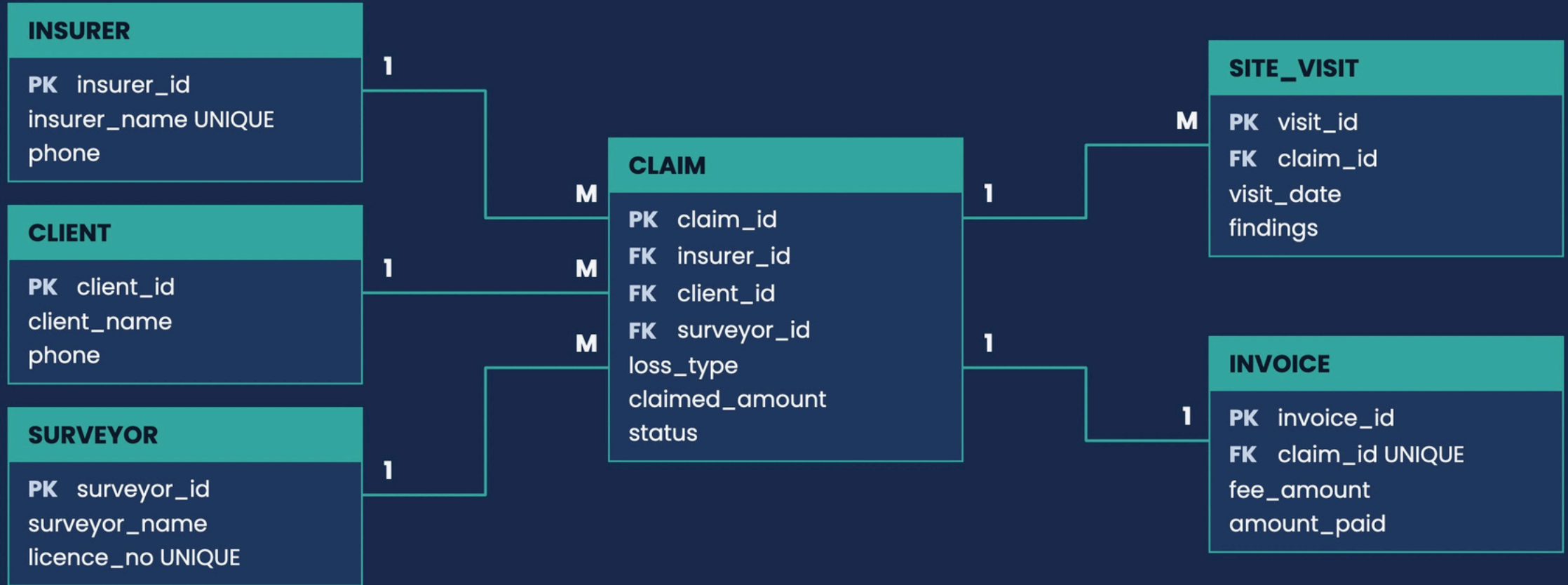
Dependency in CLAIM	Moves to a new table	New key
insurer_name → insurer_phone	INSURER (insurer_name, phone)	insurer_id
client_name → client_phone	CLIENT (client_name, phone)	client_id
surveyor_name → licence_no	SURVEYOR (surveyor_name, licence_no)	surveyor_id
Design choice: fee_amount, amount_paid	INVOICE (claim_id, fee_amount, amount_paid)	invoice_id

CLAIM NOW KEEPS ONLY

PK claim_id	FK insurer_id	FK client_id	FK surveyor_id	loss_type	claimed_amount	status
-------------	---------------	--------------	----------------	-----------	----------------	--------

INVOICE is a design choice, not a 3NF rule: fees depend directly on claim_id, but unbilled claims would leave empty columns. balance_due is never stored; a view calculates it.

Final Schema in 3NF



Also BCNF: every determinant is a key

Primary key on all 6 tables

5 foreign keys

UNIQUE claim_id on INVOICE = 1:1

CHECK amount_paid ≤ fee_amount

Anomalies fixed: a surveyor can be added with no claims, a phone changes in one row, and deleting a claim keeps the client.

Sample Data & Constraints

118 realistic dummy records. No real client data is stored.

Table	Rows
INSURER	6
CLIENT	14
SURVEYOR	6
CLAIM	30
SITE_VISIT	43
INVOICE	19
Total	118

Key SQL decisions

- **ON DELETE RESTRICT:** can't delete an insurer, client or surveyor that has claims
- **ON DELETE CASCADE:** deleting a claim removes its visits and invoice
- **UNIQUE claim_id** on INVOICE enforces 1 : 1
- **CHECK** rules on amounts and payments
- **balance_due** is calculated in a view, never stored
- Triggers write every insert, update and delete to **audit_log**

Core System Capabilities

The screenshot displays the Mangaldeep Claims system interface. The left sidebar contains navigation links: Dashboard, Claims (active), Invoices, Directory, Audit Trail, SQL Showcase, and Users. The main content area shows details for claim CLM-0003, titled 'Himal Cement Udyog'. It includes a 'Claim' section with fields for Insurer (Kailash Mutual Insurance), Client (Himal Cement Udyog), Surveyor (Bijay Rai), Loss type (Industrial), Claimed amount (Rs 62,00,000), and Site visits (3). Below this is a 'Site visits' section with a list of three visits, each with a date, description, and an 'Edit' link. The right sidebar contains a 'Status' section with a dropdown menu set to 'Surveyed' and an 'Update status' button. Below that is an 'Invoice' section with fields for Invoice (INV-0003), Fee (Rs 98,000), Paid (Rs 40,000), and Balance due (Rs 58,000). It also includes a 'Part paid' button, a 'Payment received (Rs)' field, a 'Record payment' button, and links to 'Correct invoice' and 'Delete invoice'. At the bottom, there is a 'Record a site visit' section with a 'Visit date' field (05/10/2026) and a 'Findings' field.

6
Linked tables

13
SQL reports

21
Audit triggers

2
User roles



Claim Data Flow

One claim entry links to its insurer, client and surveyor by ID. Site visits and the invoice hang off the claim, so no detail is typed twice and balances are worked out by the database.

Insurer · Client · Surveyor



Claim



Visits · Invoice



Reports

Claims Dashboard

- Open claims and unpaid fees at a glance
- Every table can show the SQL behind it
- Click a claim for its visits, invoice and change history

Mangaldeep Claims

Dashboard

Claims

Invoices

Directory

Audit Trail

SQL Showcase

Users

OA

Office Admin

Admin

Sign out

MANGALDEEP CONSULTING

Dashboard

New claim

Open claims and unpaid survey fees at a glance, read live from the MySQL database.

Open claims
11
of 15 claims in total

Value under survey
Rs 2,89,75,000
Claimed on open claims

Fees billed
Rs 2,90,000

Fees outstanding
Rs 1,40,000
4 invoices still owing

> View SQL for these figures

Open claims

Claims that are not closed yet

View all

CLAIM	CLIENT	TYPE	CLAIMED	VISITS	LAST VISIT	STATUS
CLM-0015	Anita Gurung	Motor	Rs 2,75,000	0	—	Open
CLM-0014	Pokhara Auto Works	Motor	Rs 3,80,000	1	22 Sept 2026	Open
CLM-0013	Sita Ram Shrestha	Motor	Rs 5,40,000	1	27 Aug 2026	Surveyed
CLM-0012	Narayani Textiles	Industrial	Rs 31,50,000	0	—	Open
CLM-0010	Bagmati Traders Pvt. Ltd.	Marine	Rs 13,20,000	2	20 Aug 2026	Surveyed
CLM-0009	Himal Cement Udyog	Industrial	Rs 87,50,000	1	15 Sept 2026	Open
CLM-0008	Ramesh Thapa	Motor	Rs 2,15,000	0	—	Open
CLM-0007	Pokhara Auto Works	Motor	Rs 6,90,000	2	12 Aug 2026	Surveyed
CLM-0006	Narayani Textiles	Industrial	Rs 44,80,000	1	3 Sept 2026	Open
CLM-0004	Kathmandu Freight Lines	Marine	Rs 29,75,000	2	24 Jul 2026	Surveyed
CLM-0003	Himal Cement Udyog	Industrial	Rs 62,00,000	3	19 Jul 2026	Surveyed

> View SQL

Claims by loss type

Total amount claimed

Industrial - 4 claims

Rs 2,25,80,000

Marine - 4 claims

Rs 83,85,000

Motor - 7 claims

Rs 28,35,000

> View SQL

Front End → Database (CRUD)

Every screen reads and writes the live MySQL database.

CREATE

Add

New claim, site visit, invoice, insurer, client or surveyor.

```
INSERT INTO claim ...
```

READ

View & search

Dashboard, claim list, filters, claim detail, 13 reports.

```
SELECT ... FROM v_claim_over  
view
```

UPDATE

Change

Claim status, edit visit findings, record a payment.

```
UPDATE invoice SET amount_  
paid ...
```

DELETE

Remove

Delete a visit, invoice or claim. Blocked if still in use.

```
DELETE FROM site_visit ...
```

Each table in the app has a “View SQL” button that shows the exact query behind it.

SQL Reports

13 report queries answer the business questions. A few of the techniques used:

JOIN 4 tables

Open claims with insurer, client and surveyor names

GROUP BY + SUM

Fees billed, collected and still owed per insurer

VIEW

v_unpaid_invoices lists every invoice with money owing

Subquery

Claims above the average amount

NOT IN

Claims that have no site visit yet

LEFT JOIN

Surveyor workload, including surveyors with no claims

MAX + COUNT

Latest site visit and number of visits per claim

HAVING COUNT(*) > 1

Possible duplicate claims, caught by a query

Two Reports, Explained

Q1 · Which insurers owe us the most?

Insurer	Billed	Collected	Owed
Everest Shield	2,70,000	97,000	1,73,000
Kailash Mutual	2,27,000	1,22,000	1,05,000
Gandaki	88,000	61,000	27,000

GROUP BY insurer + SUM + HAVING

→ Everest Shield owes the most (Rs 1,73,000). Follow up with them first.

Q2 · Has a claim been entered twice?

Client	Insurer	Type	Times
Pokhara Auto Works	Annapurna General	Motor	2

GROUP BY client, insurer, loss type + HAVING COUNT(*) > 1

→ Claims 7 and 24 look like the same loss. Check before sending a second surveyor.

Testing Results

#	Test	Expected	Result
1	Claim with a negative amount	Rejected by CHECK	✓ Pass
2	Claim for an insurer that doesn't exist	Rejected by FOREIGN KEY	✓ Pass
3	Pay more than the invoice fee	Rejected by CHECK	✓ Pass
4	Second invoice for the same claim	Rejected by UNIQUE	✓ Pass
5	Delete an insurer that has claims	Rejected by RESTRICT	✓ Pass
6	Delete a claim	Its visits and invoice removed (CASCADE)	✓ Pass
7	Balance due for claim 3	$98,000 - 40,000 = 58,000$	✓ Pass
8	Row counts after loading data	118 records	✓ Pass

Corrected during the project: SITE_VISIT key changed from (claim_id, visit_date) to visit_id so two visits on the same day are allowed · sample data raised from 59 to 118 rows to meet the ~100 record target.

Script: database/06_tests.sql. Live demo: run test 3 or 5 to show bad data being blocked.

Backup & Restore

A full export can be taken at any time and restored in two commands.

01 • BACKUP

Export

Tables, data, views and triggers in one SQL file.

```
mysqldump --databases mangaldeep_claims > backup.sql
```

02 • RESTORE

Import

Rebuilds the whole database on any MySQL server.

```
mysql -u <user> -p < backup.sql
```

03 • CHECK

Verified

Restored into a test database. Counts matched: 30 claims, 43 visits, 19 invoices.

```
SELECT COUNT(*) FROM claim;
```

File kept in the project folder: database/backup/mangaldeep_claims_backup.sql

Stakeholder Feedback

What he asked for

1. Track fees and payments so unpaid fees are not missed.
2. Show which claims are still open.
3. Record each site visit separately.

How we addressed it

1. An INVOICE table with fee_amount and amount_paid, plus an unpaid fees list.
2. A status column on CLAIM and an open claims list.
3. A SITE_VISIT table with one row per visit, linked to its claim.

Each point maps to the same number on the right.

Value & Limitations

Estimated yearly impact for a firm handling about 80 claims a month.

~960

claims a year in one system instead of Excel, Word and WhatsApp

~20 days

of staff time saved a year by not retyping insurer and client details

Rs 17 lakh

in unpaid fees listed in one place, so none are forgotten

~19

duplicate claims a year caught before a wasted site visit

What Mangaldeep gains

- Monthly fee report in seconds, not a day of manual totals
- Balances worked out by the database, not by hand
- Every change logged with who made it

Limitations

- Dummy data only; real Excel records not migrated yet
- No claim date yet, so overdue claims can't be tracked
- No photo or document uploads

Assumptions: 80 claims/month · average survey fee Rs 12,000 (≈ Rs 1.15 crore billed/year) · 10 min retyping per claim · 15% of fees paid late or chased · 2% duplicate entries. To be confirmed with Mangaldeep's real records.

Recommendations

- 1 Migrate the existing Excel claim records into MySQL.
- 2 Add a claim date so overdue claims can be tracked.
- 3 Standardise damage-report templates and allow photo uploads.
- 4 Block likely duplicate claims at entry, not just flag them.
- 5 Send email reminders for unpaid invoices.





Conclusion

One MySQL database replaces disconnected Excel sheets, Word files and WhatsApp threads. Six normalized tables link insurers, clients, surveyors, claims, site visits and invoices, so details are entered once.

Mangaldeep gains a live view of open claims and unpaid fees, and a full audit trail of every change.



Mangaldeep Consulting



Thank You!

Smarter claim surveys, powered by Next.js and MySQL

Questions?

Shaksham Karki and Bikram Timalisina